1 Data Structure

1.1 segment tree lazytag

```
struct SegmentTree{
    struct Node{
        int sum, f, d, len;
    } tree[MAXN << 2];
    void pull(int id){
        tree[id].sum = tree[id << 1].sum +
            tree[id << 1 | 1].sum;
    }
    void addtag(int id, int a, int d){
        tree[id].f += a;
        tree[id].d += d;
        tree[id].sum += (2 * a + (tree[id].
            len - 1) * d) * tree[id].len /
            2;
    }
    void push(int id){
        if(!tree[id].f && !tree[id].d)
            return;
        int mid_f = tree[id].f + tree[id <<
            1].len * tree[id].d;
        addtag(id << 1, tree[id].f, tree[id
            ].d);
        addtag(id << 1 | 1, mid_f, tree[id].
            d);
        tree[id].f = tree[id].d = 0;
    }
    void build(int l, int r, int id, int ar
        []){
        tree[id].len = r - l + 1;
        tree[id].f = tree[id].d = 0;
        if(l == r){
            tree[id].sum = ar[l];
            return;
        }
        int mid = (l + r) >> 1;
        build(l, mid, id << 1, ar);
        build(mid + 1, r, id << 1 | 1, ar);
        pull(id);
    }
    void update(int ql, int qr, int a, int d
        , int l, int r, int id){
        if(ql <= l and r <= qr){
            addtag(id, a + (l - ql) * d, d);
            return;
        }
        push(id);
        int mid = (l + r) >> 1;
        if(ql <= mid) update(ql, qr, a, d, l
            , mid, id << 1);
        if(qr > mid) update(ql, qr, a, d,
            mid + 1, r, id << 1 | 1);
        pull(id);
    }
    int query(int ql, int qr, int l, int r,
        int id){
        if(ql <= l and r <= qr) return tree[
            id].sum;
        push(id);
        int mid = (l + r) >> 1, re = 0;
        if(ql <= mid) re += query(ql, qr, l,
            mid, id << 1);
        if(qr > mid) re += query(ql, qr, mid
            + 1, r, id << 1 | 1);
        return re;
    }
};
```

1.2 treap

```
#include<bits/stdc++.h>
using namespace std;
#define KCC ios::sync_with_stdio(0);cin.tie
    (0);
#define pb push_back
#define pii pair<int, int>
#define F first
#define S second


mt19937 rnd(time(0));
const int MAXN = 2e5 + 5;
int left_node[MAXN], right_node[MAXN],
    priority[MAXN], sz[MAXN], node_cnt = 0,
    root = 0;
char val[MAXN];
void up(int i)
{
    sz[i] = sz[left_node[i]] + sz[right_node
        [i]] + 1;
}
void split(int i, int l, int r, int rank)
{
    if(i == 0){
        left_node[r] = right_node[l] = 0;
        return;
    }
    if(sz[left_node[i]] + 1 <= rank){
        right_node[l] = i;
        split(right_node[i], i, r, rank - sz
            [left_node[i]] - 1);
    }
    else{
        left_node[r] = i;
        split(left_node[i], l, i, rank);
    }
    up(i);
}

int merge(int l, int r)
{
    if(l == 0 or r == 0) return l + r;
    if(priority[l] >= priority[r]){
        right_node[l] = merge(right_node[l],
            r);
        up(l);
        return l;
    }
    else{
        left_node[r] = merge(l, left_node[r
            ]);
        up(r);
        return r;
    }
}
void out(int i)
{
    if(i == 0) return;
    out(left_node[i]);
    cout << val[i];
    out(right_node[i]);
}
int main()
{
    KCC
    int n, m;
    cin >> n >> m;
    for(int i = 0; i < n; i++){
        char c;
        cin >> c;
        priority[++node_cnt] = rnd();
        val[node_cnt] = c;
        sz[node_cnt] = 1;
        root = merge(root, node_cnt);
    }
    while(m--){
        int a, b;
        cin >> a >> b;

        split(root, 0, 0, b);
        int lm = right_node[0];
        int r = left_node[0];

        split(lm, 0, 0, a - 1);
        int l = right_node[0];
        int m = left_node[0];

        root = merge(merge(l, r), m);
    }
    out(root);
    return 0;
}
```

2 Flow

2.1 dinic

```
struct Dinic{
    struct Edge{
        int to, cap, rev;
    };
    vector<Edge> adj[505];
    vector<int> level, ptr;
    int n;
    Dinic(int _n){
        n = _n;
    }
    void addedge(int u, int v, int w){
        adj[u].pb({v, w, adj[v].size()});
        adj[v].pb({u, 0, adj[u].size() - 1})
            ;
    }
    bool bfs(int s, int t)
    {
        fill(level.begin(), level.end(), -1)
            ;
        level[s] = 0;
        queue<int> q;
        q.push(s);
        while(q.size()){
            int x = q.front(); q.pop();
            for(auto edge : adj[x]){
                if(edge.cap > 0 and level[
                    edge.to] == -1){
                    level[edge.to] = level[x
                        ] + 1;
                    q.push(edge.to);
                }
            }
        }
        return (level[t] != -1);
    }
    int dfs(int now, int t, int pushed)
    {
        if(now == t) return pushed;
        for(int &i = ptr[now]; i < adj[now].
            size(); i++){
            auto &w = adj[now][i];
            if(level[now] + 1 != level[w.to]
                or w.cap == 0) continue;
            int tr = dfs(w.to, t, min(pushed
                , w.cap));
            if(tr == 0) continue;
            w.cap -= tr;
            adj[w.to][w.rev].cap += tr;
            return tr;
        }
        return 0;
    }
    int compute_max_flow(){
        int max_flow = 0;
        while(bfs(1, n)){
            fill(ptr.begin(), ptr.end(), 0);
            while(int pushed = dfs(1, n, 1
                e18)){
                max_flow += pushed;
            }
        }
        return max_flow;
    }
};
```

3 Graph

3.1 SCC

```
struct SCC{
    vector<vector<int>> g, rg;
    vector<bool> vis, used;
    vector<int> component, st;
    int component_cnt = 0, n;
    void dfs1(int node){
        vis[node] = true;
        for(int i : g[node]) if(!vis[i])
            dfs1(i);
```

```
        st.pb(node);
    }
    void dfs2(int node){
        used[node] = true;
        for(int i : rg[node]) if(!used[i])
            dfs2(i);
        component[node] = component_cnt;
    }
    SCC(const vector<vector<int>> v){
        n = v.size() - 1;
        g = v;
        rg.resize(n + 1);
        vis.resize(n + 1, false);
        used.resize(n + 1, false);
        component.resize(n + 1, 0);
        for(int i = 1; i <= n; i++) for(int
            j : v[i]) rg[j].pb(i);
    };
    vector<int> find_component(){
        for(int i = 1; i <= n; i++) if(!vis[
            i]) dfs1(i);
        reverse(st.begin(), st.end());
        for(int i : st) if(!used[i]){
            component_cnt++;
            dfs2(i);
        }
        return component;
    }
};
```

4 Linear Programming

4.1 simplex

```
/*target:
  max \sum_{j=1}^n A_{0,j}*x_j
condition:
  \sum_{j=1}^n A_{i,j}*x_j <= A_{i,0} |i=1~m
  x_j >= 0 |j=1~n
VDB = vector<double>*/
template<class VDB>
VDB simplex(int m,int n,vector<VDB> a){
  vector<int> left(m+1), up(n+1);
  iota(left.begin(), left.end(), n);
  iota(up.begin(), up.end(), 0);
  auto pivot = [&](int x, int y){
    swap(left[x], up[y]);
    auto k = a[x][y]; a[x][y] = 1;
    vector<int> pos;
    for(int j = 0; j <= n; ++j){
      a[x][j] /= k;
      if(a[x][j] != 0) pos.push_back(j);
    }
    for(int i = 0; i <= m; ++i){
      if(a[i][y]==0 || i == x) continue;
      k = a[i][y], a[i][y] = 0;
      for(int j : pos) a[i][j] -= k*a[x][j];
    }
  };
  for(int x,y;;){
    for(int i=x=1; i <= m; ++i)
      if(a[i][0]<a[x][0]) x = i;
    if(a[x][0]>=0) break;
    for(int j=y=1; j <= n; ++j)
      if(a[x][j]<a[x][y]) y = j;
    if(a[x][y]>=0) return VDB();//infeasible
    pivot(x, y);
  }
  for(int x,y;;){
    for(int j=y=1; j <= n; ++j)
      if(a[0][j] > a[0][y]) y = j;
    if(a[0][y]<=0) break;
    x = -1;
    for(int i=1; i<=m; ++i) if(a[i][y] > 0)
      if(x == -1 || a[i][0]/a[i][y]
        < a[x][0]/a[x][y]) x = i;
    if(x == -1) return VDB();//unbounded
    pivot(x, y);
  }
  VDB ans(n + 1);
  for(int i = 1; i <= m; ++i)
    if(left[i] <= n) ans[left[i]] = a[i][0];
  ans[0] = -a[0][0];
  return ans;
}
```

5 Number Theory

5.1 basic

```
template<typename T>
void gcd(const T &a,const T &b,T &d,T &x,T &
    y){
  if(!b) d=a,x=1,y=0;
  else gcd(b,a%b,d,y,x), y-=x*(a/b);
}
long long int phi[N+1];
void phiTable(){
  for(int i=1;i<=N;i++)phi[i]=i;
  for(int i=1;i<=N;i++)for(x=i*2;x<=N;x+=i)
      phi[x]-=phi[i];
}
void all_divdown(const LL &n) {// all n/x
  for(LL a=1;a<=n;a=n/(n/(a+1))){
    // dosomething;
  }
}
const int MAXPRIME = 1000000;
int iscom[MAXPRIME], prime[MAXPRIME],
    primecnt;
int phi[MAXPRIME], mu[MAXPRIME];
void sieve(void){
  memset(iscom,0,sizeof(iscom));
  primecnt = 0;
  phi[1] = mu[1] = 1;
  for(int i=2;i<MAXPRIME;++i) {
    if(!iscom[i]) {
      prime[primecnt++] = i;
      mu[i] = -1;
      phi[i] = i-1;
    }
    for(int j=0;j<primecnt;++j) {
      int k = i * prime[j];
      if(k>=MAXPRIME) break;
      iscom[k] = prime[j];
      if(i%prime[j]==0) {
        mu[k] = 0;
        phi[k] = phi[i] * prime[j];
        break;
      } else {
        mu[k] = -mu[i];
        phi[k] = phi[i] * (prime[j]-1);
      }
    }
  }
}

bool g_test(const LL &g, const LL &p, const
    vector<LL> &v) {
  for(int i=0;i<v.size();++i)
    if(modexp(g,(p-1)/v[i],p)==1)
      return false;
  return true;
}
LL primitive_root(const LL &p) {
  if(p==2) return 1;
  vector<LL> v;
  Factor(p-1,v);
  v.erase(unique(v.begin(), v.end()), v.end
      ());
  for(LL g=2;g<p;++g)
    if(g_test(g,p,v))
      return g;
  puts("primitive_root NOT FOUND");
  return -1;
}
int Legendre(const LL &a, const LL &p) {
    return modexp(a%p,(p-1)/2,p); }

LL inv(const LL &a, const LL &n) {
  LL d,x,y;
  gcd(a,n,d,x,y);
  return d==1 ? (x+n)%n : -1;
}

int inv[maxN];
LL invtable(int n,LL P){
  inv[1]=1;
  for(int i=2;i<n;++i)
    inv[i]=(P-(P/i))*inv[P%i]%P;
}

LL log_mod(const LL &a, const LL &b, const
    LL &p) {
  // a ^ x = b ( mod p )
  int m=sqrt(p+.5), e=1;
  LL v=inv(modexp(a,m,p), p);
  map<LL,int> x;
  x[1]=0;
  for(int i=1;i<m;++i) {
    e = LLmul(e,a,p);
    if(!x.count(e)) x[e] = i;
  }
  for(int i=0;i<m;++i) {
    if(x.count(b)) return i*m + x[b];
    b = LLmul(b,v,p);
  }
  return -1;
}

LL Tonelli_Shanks(const LL &n, const LL &p)
    {
  // x^2 = n ( mod p )
  if(n==0) return 0;
  if(Legendre(n,p)!=1) while(1) { puts("SQRT
      ROOT does not exist"); }
  int S = 0;
  LL Q = p-1;
  while( !(Q&1) ) { Q>>=1; ++S; }
  if(S==1) return modexp(n%p,(p+1)/4,p);
  LL z = 2;
  for(;Legendre(z,p)!=-1;++z)
  LL c = modexp(z,Q,p);
  LL R = modexp(n%p,(Q+1)/2,p), t = modexp(n
      %p,Q,p);
  int M = S;
  while(1) {
    if(t==1) return R;
    LL b = modexp(c,1L<<(M-i-1),p);
    R = LLmul(R,b,p);
    t = LLmul( LLmul(b,b,p), t, p);
    c = LLmul(b,b,p);
    M = i;
  }
  return -1;
}

template<typename T>
T Euler(T n){
  T ans=n;
  for(T i=2;i*i<=n;++i){
    if(n%i==0){
      ans=ans/i*(i-1);
      while(n%i==0)n/=i;
    }
  }
  if(n>1)ans=ans/n*(n-1);
  return ans;
}

//Chinese_remainder_theorem
template<typename T>
T pow_mod(T n,T k,T m){
  T ans=1;
  for(n=(n>=m?n%m:n);k;k>>=1){
    if(k&1)ans=ans*n%m;
    n=n*n%m;
  }
  return ans;
}
template<typename T>
T crt(vector<T> &m,vector<T> &a){
  T M=1,tM,ans=0;
  for(int i=0;i<(int)m.size();++i)M*=m[i];
  for(int i=0;i<(int)a.size();++i){
    tM=M/m[i];
    ans=(ans+(a[i]*tM%M)*pow_mod(tM,Euler(m[
        i])-1,m[i])%M)%M;
    /*如果m[i]是質數，Euler(m[i])-1=m[i]-2，
        就不用算Euler了*/
  }
  return ans;
}
```

```java
//java code
//求sqrt(N)的連分數
public static void Pell(int n){
  BigInteger N,p1,p2,q1,q2,a0,a1,a2,g1,g2,h1
      ,h2,p,q;
  g1=q2=p1=BigInteger.ZERO;
  h1=q1=p2=BigInteger.ONE;
  a0=a1=BigInteger.valueOf((int)Math.sqrt
      (1.0*n));
  BigInteger ans=a0.multiply(a0);
  if(ans.equals(BigInteger.valueOf(n))){
    System.out.println("No solution!");
    return ;
  }
  while(true){
    g2=a1.multiply(h1).substract(g1);
    h2=N.substract(g2.pow(2)).divide(h1);
    a2=g2.add(a0).divide(h2);
    p=a1.multiply(p2).add(p1);
    q=a1.multiply(q2).add(q1);
    if(p.pow(2).substract(N.multiply(q.pow
        (2))).compareTo(BigInteger.ONE)==0)
        break;
    g1=g2;h1=h2;a1=a2;
    p1=p2;p2=p;
    q1=q2;q2=q;
  }
  System.out.println(p+" "+q);
}
```

5.2 bit set

```cpp
void sub_set(int S){
  int sub=S;
  do{
    //對某集合的子集合的處理
    sub=(sub-1)&S;
  }while(sub!=S);
}
void k_sub_set(int k,int n){
  int comb=(1<<k)-1,S=1<<n;
  while(comb<S){
    //對大小為k的子集合的處理
    int x=comb&-comb,y=comb+x;
    comb=((comb&~y)/x>>1)|y;
  }
}
```

5.3 cantor expansion

```cpp
int factorial[MAXN];
void init(){
  factorial[0]=1;
  for(int i=1;i<=MAXN;++i)factorial[i]=
      factorial[i-1]*i;
}
int encode(const vector<int> &s){
  int n=s.size(),res=0;
  for(int i=0;i<n;++i){
    int t=0;
    for(int j=i+1;j<n;++j)
      if(s[j]<s[i])++t;
    res+=t*factorial[n-i-1];
  }
  return res;
}
vector<int> decode(int a,int n){
  vector<int> res;
  vector<bool> vis(n,0);
  for(int i=n-1;i>=0;--i){
    int t=a/factorial[i],j;
    for(j=0;j<n;++j)
      if(!vis[j]){
        if(t==0)break;
        --t;
      }
    res.push_back(j);
    vis[j]=1;
    a%=factorial[i];
  }
  return res;
}
```

5.4 FFT

```cpp
template<typename T,typename VT=vector<
    complex<T> > >
struct FFT{
  const T pi;
  FFT(const T pi=acos((T)-1)):pi(pi){}
  unsigned bit_reverse(unsigned a,int len){
a=((a&0x55555555U)<<1)|((a&0xAAAAAAAAU)>>1);
a=((a&0x33333333U)<<2)|((a&0xCCCCCCCCU)>>2);
a=((a&0x0F0F0F0FU)<<4)|((a&0xF0F0F0F0U)>>4);
a=((a&0x00FF00FFU)<<8)|((a&0xFF00FF00U)>>8);
a=((a&0x0000FFFFU)<<16)|((a&0xFFFF0000U)
    >>16);
    return a>>(32-len);
  }
  void fft(bool is_inv,VT &in,VT &out,int N)
      {
    int bitlen=__lg(N),num=is_inv?-1:1;
    for(int i=0;i<N;++i)out[bit_reverse(i,
        bitlen)]=in[i];
    for(int step=2;step<=N;step<<=1){
      const int mh=step>>1;
      for(int i=0;i<mh;++i){
        complex<T> wi=exp(complex<T>(0,i*num
            *pi/mh));
        for(int j=i;j<N;j+=step){
          int k=j+mh;
          complex<T> u=out[j],t=wi*out[k];
          out[j]=u+t;
          out[k]=u-t;
        }
      }
    }
    if(is_inv)for(int i=0;i<N;++i)out[i]/=N;
  }
};
```

5.5 find real root

```cpp
// an*x^n + ... + a1x + a0 = 0;
int sign(double x){
  return x < -eps ? -1 : x > eps;
}

double get(const vector<double>&coef, double
    x){
  double e = 1, s = 0;
  for(auto i : coef) s += i*e, e *= x;
  return s;
}

double find(const vector<double>&coef, int n
    , double lo, double hi){
  double sign_lo, sign_hi;
  if( !(sign_lo = sign(get(coef,lo))) )
      return lo;
  if( !(sign_hi = sign(get(coef,hi))) )
      return hi;
  if(sign_lo * sign_hi > 0) return INF;
  for(int stp = 0; stp < 100 && hi - lo >
      eps; ++stp){
    double m = (lo+hi)/2.0;
    int sign_mid = sign(get(coef,m));
    if(!sign_mid) return m;
    if(sign_lo*sign_mid < 0) hi = m;
    else lo = m;
  }
  return (lo+hi)/2.0;
}

vector<double> cal(vector<double>coef, int n
    ){
  vector<double>res;
  if(n == 1){
    if(sign(coef[1])) res.pb(-coef[0]/coef
        [1]);
    return res;
  }
  vector<double>dcoef(n);
  for(int i = 0; i < n; ++i) dcoef[i] = coef
      [i+1]*(i+1);
  vector<double>droot = cal(dcoef, n-1);
  droot.insert(droot.begin(), -INF);
  droot.pb(INF);
  for(int i = 0; i+1 < droot.size(); ++i){
    double tmp = find(coef, n, droot[i],
        droot[i+1]);
    if(tmp < INF) res.pb(tmp);
  }
  return res;
}

int main () {
  vector<double>ve;
  vector<double>ans = cal(ve, n);
  // 視情況把答案 +eps，避免 -0
}
```

5.6 FWT

```cpp
vector<int> F_OR_T(vector<int> f, bool
    inverse){
  for(int i=0; (2<<i)<=f.size(); ++i)
    for(int j=0; j<f.size(); j+=2<<i)
      for(int k=0; k<(1<<i); ++k)
        f[j+k+(1<<i)] += f[j+k]*(inverse
            ?-1:1);
  return f;
}
vector<int> rev(vector<int> A) {
  for(int i=0; i<A.size(); i+=2)
    swap(A[i],A[i^(A.size()-1)]);
  return A;
}
vector<int> F_AND_T(vector<int> f, bool
    inverse){
  return rev(F_OR_T(rev(f), inverse));
}
vector<int> F_XOR_T(vector<int> f, bool
    inverse){
  for(int i=0; (2<<i)<=f.size(); ++i)
    for(int j=0; j<f.size(); j+=2<<i)
      for(int k=0; k<(1<<i); ++k){
        int u=f[j+k], v=f[j+k+(1<<i)];
        f[j+k+(1<<i)] = u-v, f[j+k] = u+v;
      }
  if(inverse) for(auto &a:f) a/=f.size();
  return f;
}
```

5.7 LinearCongruence

```cpp
pair<LL,LL> LinearCongruence(LL a[],LL b[],
    LL m[],int n) {
  // a[i]*x = b[i] ( mod m[i] )
  for(int i=0;i<n;++i) {
    LL x, y, d = extgcd(a[i],m[i],x,y);
    if(b[i]%d!=0) return make_pair(-1LL,0LL)
        ;
    m[i] /= d;
    b[i] = LLmul(b[i]/d,x,m[i]);
  }
  LL lastb = b[0], lastm = m[0];
  for(int i=1;i<n;++i) {
    LL x, y, d = extgcd(m[i],lastm,x,y);
    if((lastb-b[i])%d!=0) return make_pair
        (-1LL,0LL);
    lastb = LLmul((lastb-b[i])/d,x,(lastm/d)
        )*m[i];
    lastm = (lastm/d)*m[i];
    lastb = (lastb+b[i])%lastm;
  }
  return make_pair(lastb<0?lastb+lastm:lastb
      ,lastm);
}
```

5.8 Lucas

```
ll C(ll n, ll m, ll p){// n!/m!/(n-m)!
 if(n<m) return 0;
 return f[n]*inv(f[m],p)%p*inv(f[n-m],p)%p;
}
ll L(ll n, ll m, ll p){
 if(!m) return 1;
 return C(n%p,m%p,p)*L(n/p,m/p,p)%p;
}
ll Wilson(ll n, ll p){ // n!%p
  if(!n)return 1;
  ll res=Wilson(n/p, p);
  if((n/p)%2) return res*(p-f[n%p])%p;
  return  res*f[n%p]%p; //(p-1)!%p=-1
}
```

5.9 Matrix

```
template<typename T>
struct Matrix{
  using rt = std::vector<T>;
  using mt = std::vector<rt>;
  using matrix = Matrix<T>;
  int r,c;
  mt m;
  Matrix(int r,int c):r(r),c(c),m(r,rt(c)){}
  rt& operator[](int i){return m[i];}
  matrix operator+(const matrix &a){
    matrix rev(r,c);
    for(int i=0;i<r;++i)
      for(int j=0;j<c;++j)
        rev[i][j]=m[i][j]+a.m[i][j];
    return rev;
  }
  matrix operator-(const matrix &a){
    matrix rev(r,c);
    for(int i=0;i<r;++i)
      for(int j=0;j<c;++j)
        rev[i][j]=m[i][j]-a.m[i][j];
    return rev;
  }
  matrix operator*(const matrix &a){
    matrix rev(r,a.c);
    matrix tmp(a.c,a.r);
    for(int i=0;i<a.r;++i)
      for(int j=0;j<a.c;++j)
        tmp[j][i]=a.m[i][j];
    for(int i=0;i<r;++i)
      for(int j=0;j<a.c;++j)
        for(int k=0;k<c;++k)
          rev.m[i][j]+=m[i][k]*tmp[j][k];
    return rev;
  }
  bool inverse(){
    Matrix t(r,r+c);
    for(int y=0;y<r;y++){
      t.m[y][c+y] = 1;
      for(int x=0;x<c;++x)
        t.m[y][x]=m[y][x];
    }
    if( !t.gas() )
      return false;
    for(int y=0;y<r;y++)
      for(int x=0;x<c;++x)
        m[y][x]=t.m[y][c+x]/t.m[y][y];
    return true;
  }
  T gas(){
    vector<T> lazy(r,1);
    bool sign=false;
    for(int i=0;i<r;++i){
      if( m[i][i]==0 ){
        int j=i+1;
        while(j<r&&!m[j][i])j++;
        if(j==r)continue;
        m[i].swap(m[j]);
        sign=!sign;
      }
      for(int j=0;j<r;++j){
        if(i==j)continue;
        lazy[j]=lazy[j]*m[i][i];
        T mx=m[j][i];
        for(int k=0;k<c;++k)
          m[j][k]=m[j][k]*m[i][i]-m[i][k]*mx
              ;
      }
    }
    T det=sign?-1:1;
    for(int i=0;i<r;++i){
      det = det*m[i][i];
      det = det/lazy[i];
      for(auto &j:m[i])j/=lazy[i];
    }
    return det;
  }
};
```

5.10 MillerRobin

```
ULL LLmul(ULL a, ULL b, const ULL &mod) {
  LL ans=0;
  while(b) {
    if(b&1) {
      ans+=a;
      if(ans>=mod) ans-=mod;
    }
    a<<=1, b>>=1;
    if(a>=mod) a-=mod;
  }
  return ans;
}
ULL mod_mul(ULL a,ULL b,ULL m){
  a%=m,b%=m;/* fast for m < 2^58 */
  ULL y=(ULL)((double)a*b/m+0.5);
  ULL r=(a*b-y*m)%m;
  return r<0?r+m:r;
}
template<typename T>
T pow(T a,T b,T mod){//a^b%mod
  T ans=1;
  for(;b;a=mod_mul(a,a,mod),b>>=1)
    if(b&1)ans=mod_mul(ans,a,mod);
  return ans;
}
int sprp[3]={2,7,61};//int範圍可解
int llsprp
    [7]={2,325,9375,28178,450775,9780504,
1795265022};//至少unsigned long long範圍
template<typename T>
bool isprime(T n,int *sprp,int num){
  if(n==2)return 1;
  if(n<2||n%2==0)return 0;
  int t=0;
  T u=n-1;
  for(;u%2==0;++t)u>>=1;
  for(int i=0;i<num;++i){
    T a=sprp[i]%n;
    if(a==0||a==1||a==n-1)continue;
    T x=pow(a,u,n);
    if(x==1||x==n-1)continue;
    for(int j=0;j<t;++j){
      x=mod_mul(x,x,n);
      if(x==1)return 0;
      if(x==n-1)break;
    }
    if(x==n-1)continue;
    return 0;
  }
  return 1;
}
```

5.11 NTT

```
2615053605667*(2^18)+1,3
15*(2^27)+1,31
479*(2^21)+1,3
7*17*(2^23)+1,3
3*3*211*(2^19)+1,5
25*(2^22)+1,3
template<typename T,typename VT=vector<T> >
struct NTT{
  const T P,G;
  NTT(T p=(1<<23)*7*17+1,T g=3):P(p),G(g){}
  unsigned bit_reverse(unsigned a,int len){
    //look FFT.cpp
  }
  T pow_mod(T n,T k,T m){
    T ans=1;
    for(n=(n>=m?n%m:n);k;k>>=1){
      if(k&1)ans=ans*n%m;
      n=n*n%m;
    }
    return ans;
  }
  void ntt(bool is_inv,VT &in,VT &out,int N)
      {
    int bitlen=__lg(N);
    for(int i=0;i<N;++i)out[bit_reverse(i,
        bitlen)]=in[i];
    for(int step=2,id=1;step<=N;step<<=1,++
        id){
      T wn=pow_mod(G,(P-1)>>id,P),wi=1,u,t;
      const int mh=step>>1;
      for(int i=0;i<mh;++i){
        for(int j=i;j<N;j+=step){
          u=out[j],t=wi*out[j+mh]%P;
          out[j]=u+t;
          out[j+mh]=u-t;
          if(out[j]>=P)out[j]-=P;
          if(out[j+mh]<0)out[j+mh]+=P;
        }
        wi=wi*wn%P;
      }
    }
    if(is_inv){
      for(int i=1;i<N/2;++i)swap(out[i],out[
          N-i]);
      T invn=pow_mod(N,P-2,P);
      for(int i=0;i<N;++i)out[i]=out[i]*invn
          %P;
    }
  }
};
```

5.12 Simpson

```
double simpson(double a,double b){
  double c=a+(b-a)/2;
  return (F(a)+4*F(c)+F(b))*(b-a)/6;
}
double asr(double a,double b,double eps,
    double A){
  double c=a+(b-a)/2;
  double L=simpson(a,c),R=simpson(c,b);
  if( abs(L+R-A)<15*eps )
    return L+R+(L+R-A)/15.0;
  return asr(a,c,eps/2,L)+asr(c,b,eps/2,R);
}
double asr(double a,double b,double eps){
  return asr(a,b,eps,simpson(a,b));
}
```

5.13 外星模運算

```
//a[0]^(a[1]^a[2]^...)
#define maxn 1000000
int euler[maxn+5];
bool is_prime[maxn+5];
void init_euler(){
  is_prime[1]=1;//一不是質數
  for(int i=1;i<=maxn;i++)euler[i]=i;
  for(int i=2;i<=maxn;i++){
    if(!is_prime[i]){//是質數
      euler[i]--;
      for(int j=i<<1;j<=maxn;j+=i){
        is_prime[j]=1;
        euler[j]=euler[j]/i*(i-1);
      }
    }
  }
}
LL pow(LL a,LL b,LL mod){//a^b%mod
  LL ans=1;
  for(;b;a=a*a%mod,b>>=1)
    if(b&1)ans=ans*a%mod;
  return ans;
```

```
}
bool isless(LL *a,int n,int k){
  if(*a==1)return k>1;
  if(--n==0)return *a<k;
  int next=0;
  for(LL b=1;b<k;++next)
    b*=*a;
  return isless(a+1,n,next);
}
LL high_pow(LL *a,int n,LL mod){
  if(*a==1||--n==0)return *a%mod;
  int k=0,r=euler[mod];
  for(LL tma=1;tma!=pow(*a,k+r,mod);++k)
    tma=tma*(*a)%mod;
  if(isless(a+1,n,k))return pow(*a,high_pow(
      a+1,n,k),mod);
  int tmd=high_pow(a+1,n,r), t=(tmd-k+r)%r;
  return pow(*a,k+t,mod);
}
LL a[1000005];
int t,mod;
int main(){
  init_euler();
  scanf("%d",&t);
  #define n 4
  while(t--){
    for(int i=0;i<n;++i)scanf("%lld",&a[i]);
    scanf("%d",&mod);
    printf("%lld\n",high_pow(a,n,mod));
  }
  return 0;
}
```

5.14 數位統計

```
ll d[65], dp[65][2];//up區間是不是完整
ll dfs(int p,bool is8,bool up){
  if(!p)return 1; // 回傳0是不是答案
  if(!up&&~dp[p][is8])return dp[p][is8];
  int mx = up?d[p]:9;//可以用的有那些
  ll ans=0;
  for(int i=0;i<=mx;++i){
    if( is8&&i==7 )continue;
    ans += dfs(p-1,i==8,up&&i==mx);
  }
  if(!up)dp[p][is8]=ans;
  return ans;
}
ll f(ll N){
  int k=0;
  while(N){  // 把數字先分解到陣列
    d[++k] = N%10;
    N/=10;
  }
  return dfs(k,false,true);
}
```

5.15 質因數分解

```
LL func(const LL n,const LL mod,const int c)
    {
  return (LLmul(n,n,mod)+c+mod)%mod;
}

LL pollorrho(const LL n, const int c) {//循
    環節長度
  LL a=1, b=1;
  a=func(a,n,c)%n;
  b=func(b,n,c)%n; b=func(b,n,c)%n;
  while(gcd(abs(a-b),n)==1) {
    a=func(a,n,c)%n;
    b=func(b,n,c)%n; b=func(b,n,c)%n;
  }
  return gcd(abs(a-b),n);
}

void prefactor(LL &n, vector<LL> &v) {
  for(int i=0;i<12;++i) {
    while(n%prime[i]==0) {
      v.push_back(prime[i]);
      n/=prime[i];
    }
  }
}

void smallfactor(LL n, vector<LL> &v) {
  if(n<MAXPRIME) {
    while(isp[(int)n]) {
      v.push_back(isp[(int)n]);
      n/=isp[(int)n];
    }
    v.push_back(n);
  } else {
    for(int i=0;i<primecnt&&prime[i]*prime[i
        ]<=n;++i) {
      while(n%prime[i]==0) {
        v.push_back(prime[i]);
        n/=prime[i];
      }
    }
    if(n!=1) v.push_back(n);
  }
}

void comfactor(const LL &n, vector<LL> &v) {
  if(n<1e9) {
    smallfactor(n,v);
    return;
  }
  if(Isprime(n)) {
    v.push_back(n);
    return;
  }
  LL d;
  for(int c=3;;++c) {
    d = pollorrho(n,c);
    if(d!=n) break;
  }
  comfactor(d,v);
  comfactor(n/d,v);
}

void Factor(const LL &x, vector<LL> &v) {
  LL n = x;
  if(n==1) { puts("Factor 1"); return; }
  prefactor(n,v);
  if(n==1) return;
  comfactor(n,v);
  sort(v.begin(),v.end());
}

void AllFactor(const LL &n,vector<LL> &v) {
  vector<LL> tmp;
  Factor(n,tmp);
  v.clear();
  v.push_back(1);
  int len;
  LL now=1;
  for(int i=0;i<tmp.size();++i) {
    if(i==0 || tmp[i]!=tmp[i-1]) {
      len = v.size();
      now = 1;
    }
    now*=tmp[i];
    for(int j=0;j<len;++j)
      v.push_back(v[j]*now);
  }
}
```

6 String

6.1 AC 自動機

```
template<char L='a',char R='z'>
class ac_automaton{
  struct joe{
    int next[R-L+1],fail,efl,ed,cnt_dp,vis;
    joe():ed(0),cnt_dp(0),vis(0){
      for(int i=0;i<=R-L;++i)next[i]=0;
    }
  };
public:
  std::vector<joe> S;
  std::vector<int> q;
  int qs,qe,vt;
  ac_automaton():S(1),qs(0),qe(0),vt(0){}
  void clear(){
    q.clear();
    S.resize(1);
    for(int i=0;i<=R-L;++i)S[0].next[i]=0;
    S[0].cnt_dp=S[0].vis=qs=qe=vt=0;
  }
  void insert(const char *s){
    int o=0;
    for(int i=0,id;s[i];++i){
      id=s[i]-L;
      if(!S[o].next[id]){
        S.push_back(joe());
        S[o].next[id]=S.size()-1;
      }
      o=S[o].next[id];
    }
    ++S[o].ed;
  }
  void build_fail(){
    S[0].fail=S[0].efl=-1;
    q.clear();
    q.push_back(0);
    ++qe;
    while(qs!=qe){
      int pa=q[qs++],id,t;
      for(int i=0;i<=R-L;++i){
        t=S[pa].next[i];
        if(!t)continue;
        id=S[pa].fail;
        while(~id&&!S[id].next[i])id=S[id].
            fail;
        S[t].fail=~id?S[id].next[i]:0;
        S[t].efl=S[S[t].fail].ed?S[t].fail:S
            [S[t].fail].efl;
        q.push_back(t);
        ++qe;
      }
    }
  }
  /*DP出每個前綴在字串s出現的次數並傳回所有
      字串被s匹配成功的次數O(N+M)*/
  int match_0(const char *s){
    int ans=0,id,p=0,i;
    for(i=0;s[i];++i){
      id=s[i]-L;
      while(!S[p].next[id]&&p)p=S[p].fail;
      if(!S[p].next[id])continue;
      p=S[p].next[id];
      ++S[p].cnt_dp;/*匹配成功則它所有後綴都
          可以被匹配(DP計算)*/
    }
    for(i=qe-1;i>=0;--i){
      ans+=S[q[i]].cnt_dp*S[q[i]].ed;
      if(~S[q[i]].fail)S[S[q[i]].fail].
          cnt_dp+=S[q[i]].cnt_dp;
    }
    return ans;
  }
  /*多串匹配走efl邊並傳回所有字串被s匹配成功
      的次數O(N*M^1.5)*/
  int match_1(const char *s)const{
    int ans=0,id,p=0,t;
    for(int i=0;s[i];++i){
      id=s[i]-L;
      while(!S[p].next[id]&&p)p=S[p].fail;
      if(!S[p].next[id])continue;
      p=S[p].next[id];
      if(S[p].ed)ans+=S[p].ed;
      for(t=S[p].efl;~t;t=S[t].efl){
        ans+=S[t].ed;/*因為都走efl邊所以保證
            匹配成功*/
      }
    }
    return ans;
  }
  /*枚舉(s的子字串nA)的所有相異字串各恰一次
      並傳回次數O(N*M^(1/3))*/
  int match_2(const char *s){
    int ans=0,id,p=0,t;
    ++vt;
    /*把戳記vt+=1，只要vt沒溢位，所有S[p].
        vis==vt就會變成false
    這種利用vt的方法可以O(1)歸零vis陣列*/
```

```
    for(int i=0;s[i];++i){
      id=s[i]-L;
      while(!S[p].next[id]&&p)p=S[p].fail;
      if(!S[p].next[id])continue;
      p=S[p].next[id];
      if(S[p].ed&&S[p].vis!=vt){
        S[p].vis=vt;
        ans+=S[p].ed;
      }
      for(t=S[p].efl;~t&&S[t].vis!=vt;t=S[t
          ].efl){
        S[t].vis=vt;
        ans+=S[t].ed;/*因為都走efl邊所以保證
            匹配成功*/
      }
    }
    return ans;
  }
  /*把AC自動機變成真的自動機*/
  void evolution(){
    for(qs=1;qs!=qe;){
      int p=q[qs++];
      for(int i=0;i<=R-L;++i)
        if(S[p].next[i]==0)S[p].next[i]=S[S[
            p].fail].next[i];
    }
  }
};
```

6.2 hash

```
#define MAXN 1000000
#define mod 1073676287
/*mod 必須要是質數*/
typedef long long T;
char s[MAXN+5];
T h[MAXN+5];/*hash陣列*/
T h_base[MAXN+5];/*h_base[n]=(prime^n)%mod*/
void hash_init(int len,T prime){
  h_base[0]=1;
  for(int i=1;i<=len;++i){
    h[i]=(h[i-1]*prime+s[i-1])%mod;
    h_base[i]=(h_base[i-1]*prime)%mod;
  }
}
T get_hash(int l,int r){/*閉區間寫法，設編號
    為0 ~ len-1*/
  return (h[r+1]-(h[l]*h_base[r-l+1])%mod+
      mod)%mod;
}
```

6.3 KMP

```
/*產生fail function*/
void kmp_fail(char *s,int len,int *fail){
  int id=-1;
  fail[0]=-1;
  for(int i=1;i<len;++i){
    while(~id&&s[id+1]!=s[i])id=fail[id];
    if(s[id+1]==s[i])++id;
    fail[i]=id;
  }
}
/*以字串B匹配字串A，傳回匹配成功的數量(用B的
    fail)*/
int kmp_match(char *A,int lenA,char *B,int
    lenB,int *fail){
  int id=-1,ans=0;
  for(int i=0;i<lenA;++i){
    while(~id&&B[id+1]!=A[i])id=fail[id];
    if(B[id+1]==A[i])++id;
    if(id==lenB-1){/*匹配成功*/
      ++ans, id=fail[id];
    }
  }
  return ans;
}
```

6.4 manacher

```
//原字串: asdsasdsa
//先把字串變成這樣: @#a#s#d#s#a#s#d#s#a#
void manacher(char *s,int len,int *z){
  int l=0,r=0;
  for(int i=1;i<len;++i){
    z[i]=r>i?min(z[2*l-i],r-i):1;
    while(s[i+z[i]]==s[i-z[i]])++z[i];
    if(z[i]+i>r)r=z[i]+i,l=i;
  }//ans = max(z)-1
}
```

6.5 minimal string rotation

```
int min_string_rotation(const string &s){
  int n=s.size(),i=0,j=1,k=0;
  while(i<n&&j<n&&k<n){
    int t=s[(i+k)%n]-s[(j+k)%n];
    ++k;
    if(t){
      if(t>0)i+=k;
      else j+=k;
      if(i==j)++j;
      k=0;
    }
  }
  return min(i,j);//最小循環表示法起始位置
}
```

6.6 reverseBWT

```
const int MAXN = 305, MAXC = 'Z';
int ranks[MAXN], tots[MAXC], first[MAXC];
void rankBWT(const string &bw){
  memset(ranks,0,sizeof(int)*bw.size());
  memset(tots,0,sizeof(tots));
  for(size_t i=0;i<bw.size();++i)
    ranks[i] = tots[int(bw[i])]++;
}
void firstCol(){
  memset(first,0,sizeof(first));
  int totc = 0;
  for(int c='A';c<='Z';++c){
    if(!tots[c]) continue;
    first[c] = totc;
    totc += tots[c];
  }
}
string reverseBwt(string bw,int begin){
  rankBWT(bw), firstCol();
  int i = begin; //原字串最後一個元素的位置
  string res;
  do{
    char c = bw[i];
    res = c + res;
    i = first[int(c)] + ranks[i];
  }while( i != begin );
  return res;
}
```

6.7 suffix array lcp

```
#define radix_sort(x,y){\
  for(i=0;i<A;++i)c[i]=0;\
  for(i=0;i<n;++i)c[x[y[i]]]++;\
  for(i=1;i<A;++i)c[i]+=c[i-1];\
  for(i=n-1;~i;--i)sa[--c[x[y[i]]]]=y[i];\
}
#define AC(r,a,b)\
  r[a]!=r[b]||a+k>=n||r[a+k]!=r[b+k]
void suffix_array(const char *s,int n,int *
    sa,int *rank,int *tmp,int *c){
  int A='z'+1,i,k,id=0;
  for(i=0;i<n;++i)rank[tmp[i]=i]=s[i];
  radix_sort(rank,tmp);
  for(k=1;id<n-1;k<<=1){
    for(id=0,i=n-k;i<n;++i)tmp[id++]=i;
    for(i=0;i<n;++i)
      if(sa[i]>=k)tmp[id++]=sa[i]-k;
    radix_sort(rank,tmp);
    swap(rank,tmp);
    for(rank[sa[0]]=id=0,i=1;i<n;++i)
      rank[sa[i]]=id+=AC(tmp,sa[i-1],sa[i]);
    A=id+1;
  }
}
//h:高度數組 sa:後綴數組 rank:排名
void suffix_array_lcp(const char *s,int len,
    int *h,int *sa,int *rank){
  for(int i=0;i<len;++i)rank[sa[i]]=i;
  for(int i=0,k=0;i<len;++i){
    if(rank[i]==0)continue;
    if(k)--k;
    while(s[i+k]==s[sa[rank[i]-1]+k])++k;
    h[rank[i]]=k;
  }
  h[0]=0;// h[k]=lcp(sa[k],sa[k-1]);
}
```

6.8 Z

```
void z_alg(char *s,int len,int *z){
  int l=0,r=0;
  z[0]=len;
  for(int i=1;i<len;++i){
    z[i]=i>r?0:(i-l+z[i-l]<z[l]?z[i-l]:r-i
        +1);
    while(i+z[i]<len&&s[i+z[i]]==s[z[i]])++z
        [i];
    if(i+z[i]-1>r)r=i+z[i]-1,l=i;
  }
}
```

7 Tarjan

7.1 dominator tree

```
struct dominator_tree{
  static const int MAXN=5005;
  int n;// 1-base
  vector<int> G[MAXN], rG[MAXN];
  int pa[MAXN], dfn[MAXN], id[MAXN], dfnCnt;
  int semi[MAXN], idom[MAXN], best[MAXN];
  vector<int> tree[MAXN]; // tree here
  void init(int _n){
    n = _n;
    for(int i=1; i<=n; ++i)
      G[i].clear(), rG[i].clear();
  }
  void add_edge(int u, int v){
    G[u].push_back(v);
    rG[v].push_back(u);
  }
  void dfs(int u){
    id[dfn[u]=++dfnCnt]=u;
    for(auto v:G[u]) if(!dfn[v])
      dfs(v),pa[dfn[v]]=dfn[u];
  }
  int find(int y,int x){
    if(y <= x) return y;
    int tmp = find(pa[y],x);
    if(semi[best[y]] > semi[best[pa[y]]])
      best[y] = best[pa[y]];
    return pa[y] = tmp;
  }
  void tarjan(int root){
    dfnCnt = 0;
    for(int i=1; i<=n; ++i){
      dfn[i] = idom[i] = 0;
      tree[i].clear();
      best[i] = semi[i] = i;
    }
    dfs(root);
    for(int i=dfnCnt; i>1; --i){
      int u = id[i];
      for(auto v:rG[u]) if(v=dfn[v]){
        find(v,i);
        semi[i]=min(semi[i],semi[best[v]]);
      }
```

```
      tree[semi[i]].push_back(i);
      for(auto v:tree[pa[i]]){
        find(v, pa[i]);
        idom[v] = semi[best[v]]==pa[i]
            ? pa[i] : best[v];
      }
      tree[pa[i]].clear();
    }
    for(int i=2; i<=dfnCnt; ++i){
      if(idom[i] != semi[i])
        idom[i] = idom[idom[i]];
      tree[id[idom[i]]].push_back(id[i]);
    }
  }
}dom;
```

7.2 tnfshb017 2 sat

```
#include<bits/stdc++.h>
using namespace std;
#define MAXN 8001
#define MAXN2 MAXN*4
#define n(X) ((X)+2*N)
vector<int>  v[MAXN2], rv[MAXN2], vis_t;
int N,M;
void addedge(int s,int e){
  v[s].push_back(e);
  rv[e].push_back(s);
}
int scc[MAXN2];
bool vis[MAXN2]={false};
void dfs(vector<int> *uv,int n,int k=-1){
  vis[n]=true;
  for(int i=0;i<uv[n].size();++i)
    if(!vis[uv[n][i]])
      dfs(uv,uv[n][i],k);
  if(uv==v)vis_t.push_back(n);
  scc[n]=k;
}
void solve(){
  for(int i=1;i<=N;++i){
    if(!vis[i])dfs(v,i);
    if(!vis[n(i)])dfs(v,n(i));
  }
  memset(vis,0,sizeof(vis));
  int c=0;
  for(int i=vis_t.size()-1;i>=0;--i)
    if(!vis[vis_t[i]])
      dfs(rv,vis_t[i],c++);
}
int main(){
  int a,b;
  scanf("%d%d",&N,&M);
  for(int i=1;i<=N;++i){
    // (A or B)&(!A & !B) A^B
    a=i*2-1;
    b=i*2;
    addedge(n(a),b);
    addedge(n(b),a);
    addedge(a,n(b));
    addedge(b,n(a));
  }
  while(M--){
    scanf("%d%d",&a,&b);
    a = a>0?a*2-1:-a*2;
    b = b>0?b*2-1:-b*2;
    // A or B
    addedge(n(a),b);
    addedge(n(b),a);
  }
  solve();
  bool check=true;
  for(int i=1;i<=2*N;++i)
    if(scc[i]==scc[n(i)])
      check=false;
  if(check){
    printf("%d\n",N);
    for(int i=1;i<=2*N;i+=2){
      if(scc[i]>scc[i+2*N]) putchar('+');
      else putchar('-');
    }
    puts("");
  }else puts("0");
  return 0;
}
```

7.3 橋連通分量

```
#define N 1005
struct edge{
  int u,v;
  bool is_bridge;
  edge(int u=0,int v=0):u(u),v(v),is_bridge
      (0){}
};
vector<edge> E;
vector<int> G[N];// 1-base
int low[N],vis[N],Time;
int bcc_id[N],bridge_cnt,bcc_cnt;// 1-base
int st[N],top;//BCC用
void add_edge(int u,int v){
  G[u].push_back(E.size());
  E.emplace_back(u,v);
  G[v].push_back(E.size());
  E.emplace_back(v,u);
}
void dfs(int u,int re=-1){//u當前點，re為u連
    接前一個點的邊
  int v;
  low[u]=vis[u]=++Time;
  st[top++]=u;
  for(int e:G[u]){
    v=E[e].v;
    if(!vis[v]){
      dfs(v,e^1);//e^1反向邊
      low[u]=min(low[u],low[v]);
      if(vis[u]<low[v]){
        E[e].is_bridge=E[e^1].is_bridge=1;
        ++bridge_cnt;
      }
    }else if(vis[v]<vis[u]&&e!=re)
      low[u]=min(low[u],vis[v]);
  }
  if(vis[u]==low[u]){//處理BCC
    ++bcc_cnt;// 1-base
    do bcc_id[v=st[--top]]=bcc_cnt;//每個點
        所在的BCC
    while(v!=u);
  }
}
void bcc_init(int n){
  Time=bcc_cnt=bridge_cnt=top=0;
  E.clear();
  for(int i=1;i<=n;++i){
    G[i].clear();
    vis[i]=bcc_id[i]=0;
  }
}
```

7.4 雙連通分量 & 割點

```
#define N 1005
vector<int> G[N];// 1-base
vector<int> bcc[N];//存每塊雙連通分量的點
int low[N],vis[N],Time;
int bcc_id[N],bcc_cnt;// 1-base
bool is_cut[N];//是否為割點
int st[N],top;
void dfs(int u,int pa=-1){//u當前點，pa父親
  int t, child=0;
  low[u]=vis[u]=++Time;
  st[top++]=u;
  for(int v:G[u]){
    if(!vis[v]){
      dfs(v,u),++child;
      low[u]=min(low[u],low[v]);
      if(vis[u]<=low[v]){
        is_cut[u]=1;
        bcc[++bcc_cnt].clear();
        do{
          bcc_id[t=st[--top]]=bcc_cnt;
          bcc[bcc_cnt].push_back(t);
        }while(t!=v);
        bcc_id[u]=bcc_cnt;
        bcc[bcc_cnt].push_back(u);
      }
    }else if(vis[v]<vis[u]&&v!=pa)//反向邊
      low[u] = min(low[u],vis[v]);
  }//u是dfs樹的根要特判
  if(pa==-1&&child<2)is_cut[u]=0;
}
void bcc_init(int n){
  Time=bcc_cnt=top=0;
  for(int i=1;i<=n;++i){
    G[i].clear();
    is_cut[i]=vis[i]=bcc_id[i]=0;
  }
}
```

8 Tree Problem

8.1 HeavyLight

```
#include<vector>
#define MAXN 100005
int siz[MAXN],max_son[MAXN],pa[MAXN],dep[
    MAXN];
int link_top[MAXN],link[MAXN],cnt;
vector<int> G[MAXN];
void find_max_son(int u){
  siz[u]=1;
  max_son[u]=-1;
  for(auto v:G[u]){
    if(v==pa[u])continue;
    pa[v]=u;
    dep[v]=dep[u]+1;
    find_max_son(v);
    if(max_son[u]==-1||siz[v]>siz[max_son[u
        ]])max_son[u]=v;
    siz[u]+=siz[v];
  }
}
void build_link(int u,int top){
  link[u]=++cnt;
  link_top[u]=top;
  if(max_son[u]==-1)return;
  build_link(max_son[u],top);
  for(auto v:G[u]){
    if(v==max_son[u]||v==pa[u])continue;
    build_link(v,v);
  }
}
int find_lca(int a,int b){
  //求LCA，可以在過程中對區間進行處理
  int ta=link_top[a],tb=link_top[b];
  while(ta!=tb){
    if(dep[ta]<dep[tb]){
      swap(ta,tb);
      swap(a,b);
    }
    //這裡可以對a所在的鏈做區間處理
    //區間為(link[ta],link[a])
    ta=link_top[a=pa[ta]];
  }
  //最後a,b會在同一條鏈，若a!=b還要在進行一
      次區間處理
  return dep[a]<dep[b]?a:b;
}
```

8.2 LCA

```
const int MAXN=100000; // 1-base
const int MLG=17; //log2(MAXN)+1;
int pa[MLG+2][MAXN+5];
int dep[MAXN+5];
vector<int> G[MAXN+5];
void dfs(int x,int p=0){//dfs(root);
  pa[0][x]=p;
  for(int i=0;i<=MLG;++i)
    pa[i+1][x]=pa[i][pa[i][x]];
  for(auto &i:G[x]){
    if(i==p)continue;
    dep[i]=dep[x]+1;
    dfs(i,x);
  }
```

```
}
inline int jump(int x,int d){
  for(int i=0;i<=MLG;++i)
    if((d>>i)&1) x=pa[i][x];
  return x;
}
inline int find_lca(int a,int b){
  if(dep[a]>dep[b])swap(a,b);
  b=jump(b,dep[b]-dep[a]);
  if(a==b)return a;
  for(int i=MLG;i>=0;--i){
    if(pa[i][a]!=pa[i][b]){
      a=pa[i][a];
      b=pa[i][b];
    }
  }
  return pa[0][a];
}
```

8.3 link cut tree

```
struct splay_tree{
  int ch[2],pa;//子節點跟父母
  bool rev;//反轉的懶惰標記
  splay_tree():pa(0),rev(0){ch[0]=ch[1]=0;}
};
vector<splay_tree> nd;
//有的時候用vector會TLE，要注意
//這邊以node[0]作為null節點
bool isroot(int x){//判斷是否為這棵splay
    tree的根
  return nd[nd[x].pa].ch[0]!=x&&nd[nd[x].pa
      ].ch[1]!=x;
}
void down(int x){//懶惰標記下推
  if(nd[x].rev){
    if(nd[x].ch[0])nd[nd[x].ch[0]].rev^=1;
    if(nd[x].ch[1])nd[nd[x].ch[1]].rev^=1;
    swap(nd[x].ch[0],nd[x].ch[1]);
    nd[x].rev=0;
  }
}
void push_down(int x){//所有祖先懶惰標記下推
  if(!isroot(x))push_down(nd[x].pa);
  down(x);
}
void up(int x){}//將子節點的資訊向上更新
void rotate(int x){//旋轉，會自行判斷轉的方
    向
  int y=nd[x].pa,z=nd[y].pa,d=(nd[y].ch[1]==
      x);
  nd[x].pa=z;
  if(!isroot(y))nd[z].ch[nd[z].ch[1]==y]=x;
  nd[y].ch[d]=nd[x].ch[d^1];
  nd[nd[y].ch[d]].pa=y;
  nd[y].pa=x,nd[x].ch[d^1]=y;
  up(y),up(x);
}
void splay(int x){//將x伸展到splay tree的根
  push_down(x);
  while(!isroot(x)){
    int y=nd[x].pa;
    if(!isroot(y)){
      int z=nd[y].pa;
      if((nd[z].ch[0]==y)^(nd[y].ch[0]==x))
          rotate(y);
      else rotate(x);
    }
    rotate(x);
  }
}
int access(int x){
  int last=0;
  while(x){
    splay(x);
    nd[x].ch[1]=last;
    up(x);
    last=x;
    x=nd[x].pa;
  }
  return last;//access後splay tree的根
}
void access(int x,bool is=0){//is=0就是一般
    的access
  int last=0;
  while(x){
    splay(x);
    if(is&&!nd[x].pa){
      //printf("%d\n",max(nd[last].ma,nd[nd[
          x].ch[1]].ma));
    }
    nd[x].ch[1]=last;
    up(x);
    last=x;
    x=nd[x].pa;
  }
}
void query_edge(int u,int v){
  access(u);
  access(v,1);
}
void make_root(int x){
  access(x),splay(x);
  nd[x].rev^=1;
}
void make_root(int x){
  nd[access(x)].rev^=1;
  splay(x);
}
void cut(int x,int y){
  make_root(x);
  access(y);
  splay(y);
  nd[y].ch[0]=0;
  nd[x].pa=0;
}
void cut_parents(int x){
  access(x);
  splay(x);
  nd[nd[x].ch[0]].pa=0;
  nd[x].ch[0]=0;
}
void link(int x,int y){
  make_root(x);
  nd[x].pa=y;
}
int find_root(int x){
  x=access(x);
  while(nd[x].ch[0])x=nd[x].ch[0];
  splay(x);
  return x;
}
int query(int u,int v){
//傳回uv路徑splay tree的根結點
//這種寫法無法求LCA
  make_root(u);
  return access(v);
}
int query_lca(int u,int v){
//假設求鏈上點權的總和，sum是子樹的權重和，
    data是節點的權重
  access(u);
  int lca=access(v);
  splay(u);
  if(u==lca){
    //return nd[lca].data+nd[nd[lca].ch[1]].
        sum
  }else{
    //return nd[lca].data+nd[nd[lca].ch[1]].
        sum+nd[u].sum
  }
}
struct EDGE{
  int a,b,w;
}e[10005];
int n;
vector<pair<int,int>> G[10005];
//first表示子節點，second表示邊的編號
int pa[10005],edge_node[10005];
//pa是父母節點，暫存用的，edge_node是每個編
    被存在哪個點裡面的陣列
void bfs(int root){
//在建構的時候把每個點都設成一個splay tree
  queue<int > q;
  for(int i=1;i<=n;++i)pa[i]=0;
  q.push(root);
  while(q.size()){
    int u=q.front();
    q.pop();
    for(auto P:G[u]){
      int v=P.first;
      if(v!=pa[u]){
        pa[v]=u;
        nd[v].pa=u;
        nd[v].data=e[P.second].w;
        edge_node[P.second]=v;
        up(v);
        q.push(v);
      }
    }
  }
}
void change(int x,int b){
  splay(x);
  //nd[x].data=b;
  up(x);
}
```

8.4 POJ tree

```
#include<bits/stdc++.h>
using namespace std;
#define MAXN 10005
int n,k;
vector<pair<int,int> >g[MAXN];
int size[MAXN];
bool vis[MAXN];
inline void init(){
  for(int i=0;i<=n;++i){
    g[i].clear();
    vis[i]=0;
  }
}
void get_dis(vector<int> &dis,int u,int pa,
    int d){
  dis.push_back(d);
  for(size_t i=0;i<g[u].size();++i){
    int v=g[u][i].first,w=g[u][i].second;
    if(v!=pa&&!vis[v])get_dis(dis,v,u,d+w);
  }
}
vector<int> dis;//這東西如果放在函數裡會TLE
int cal(int u,int d){
  dis.clear();
  get_dis(dis,u,-1,d);
  sort(dis.begin(),dis.end());
  int l=0,r=dis.size()-1,res=0;
  while(l<r){
    while(l<r&&dis[l]+dis[r]>k)--r;
    res+=r-(l++);
  }
  return res;
}
pair<int,int> tree_centroid(int u,int pa,
    const int sz){
  size[u]=1;//找樹重心，second是重心
  pair<int,int> res(INT_MAX,-1);
  int ma=0;
  for(size_t i=0;i<g[u].size();++i){
    int v=g[u][i].first;
    if(v==pa||vis[v])continue;
    res=min(res,tree_centroid(v,u,sz));
    size[u]+=size[v];
    ma=max(ma,size[v]);
  }
  ma=max(ma,sz-size[u]);
  return min(res,make_pair(ma,u));
}
int tree_DC(int u,int sz){
  int center=tree_centroid(u,-1,sz).second;
  int ans=cal(center,0);
  vis[center]=1;
  for(size_t i=0;i<g[center].size();++i){
    int v=g[center][i].first,w=g[center][i].
        second;
    if(vis[v])continue;
    ans-=cal(v,w);
    ans+=tree_DC(v,size[v]);
  }
  return ans;
}
int main(){
  while(scanf("%d%d",&n,&k),n||k){
```

```
    init();
    for(int i=1;i<n;++i){
      int u,v,w;
      scanf("%d%d%d",&u,&v,&w);
      g[u].push_back(make_pair(v,w));
      g[v].push_back(make_pair(u,w));
    }
    printf("%d\n",tree_DC(1,n));
  }
  return 0;
}
```

9 compare

9.1 check

```
#include<bits/stdc++.h>
using namespace std;
int main() {
    // 1. 在迴圈外編譯，並檢查是否成功
    cout << "Compiling files..." << endl;

    // system() 回傳 0 代表成功執行
    int c1 = system("g++ -Wall -std=c++20
         sol.cpp -o sol.exe");
    int c2 = system("g++ -Wall -std=c++20 bf
         .cpp -o bf.exe");
    int c3 = system("g++ -Wall -std=c++20
         gen.cpp -o gen.exe");

    if (c1 != 0 || c2 != 0 || c3 != 0) {
        cout << "Compilation Failed! Check
             if sol.cpp, bf.cpp, and gen.cpp
             exist." << endl;
        return 1;
    }

    cout << "Compilation successful.
         Starting stress test..." << endl;

    for (int T = 1; ; T++) {
        // 2. 執行程序
        system("gen.exe > input.txt");
        system("sol.exe < input.txt > sol.
             out");
        system("bf.exe < input.txt > bf.out"
             );

        // 3. 使用 fc /W 比對
        if (system("fc sol.out bf.out > nul"
             )) {
            system("fc sol.out bf.out");
            cout << "WA on Test #" << T << "
                 !" << endl;
            // 發生錯誤時停止，此時 input.
                 txt 就是出錯的測資
            break;
        } else {
            if (T % 10 == 0) cout << "Passed
                 " << T << " tests..." <<
                 endl;
        }
    }
    return 0;
}
```

9.2 gen

```
#include<bits/stdc++.h>
using namespace std;

// 使用隨機種子，確保每次生成的數據都不同
mt19937_64 rng(chrono::steady_clock::now().
     time_since_epoch().count());

// 生成 [l, r] 範圍內的隨機整數
long long rnd(long long l, long long r) {
    return uniform_int_distribution<long
         long>(l, r)(rng);
}

// 生成一棵隨機樹 (N 個點，N-1 條邊)
void gen_tree(int n) {
    for (int i = 2; i <= n; i++) {
        // 讓每個點 i 連向 [1, i-1] 中的隨機
             點
        int u = i;
        int v = rnd(1, i - 1);
        cout << u << " " << v << "\n";
    }
}
```

10 default

10.1 debug

```
#ifdef DEBUG
#define dbg(...) {\
  fprintf(stderr,"%s - %d : (%s) = ",
       __PRETTY_FUNCTION__,__LINE__,#
       __VA_ARGS__);\
  _DO(__VA_ARGS__);\
}
template<typename I> void _DO(I&&x){cerr<<x
     <<endl;}
template<typename I,typename...T> void _DO(I
     &&x,T&&...tail){cerr<<x<<", ";_DO(tail
     ...);}
#else
#define dbg(...)
#endif
```

10.2 ext

```
#include<bits/extc++.h>
#include<ext/pd_ds/assoc_container.hpp>
#include<ext/pd_ds/tree_policy.hpp>
using namespace  __gnu_cxx;
using namespace  __gnu_pbds;
template<typename T>
using pbds_set = tree<T,null_type,less<T>,
     rb_tree_tag,
     tree_order_statistics_node_update>;
template<typename T,typename U>
using pbds_map = tree<T,U,less<T>,
     rb_tree_tag,
     tree_order_statistics_node_update>;
using heap=__gnu_pbds::priority_queue<int>;
//s.find_by_order(1);//0 base
//s.order_of_key(1);
```

10.3 IncStack

```
//Magic
#pragma GCC optimize "Ofast"
//stack resize,change esp to rsp if 64-bit
     system
asm("mov %0,%%esp\n" ::"g"(mem+10000000));
-Wl,--stack,214748364 -trigraphs
#pragma comment(linker, "/STACK
     :1024000000,1024000000")
//linux stack resize
#include<sys/resource.h>
void increase_stack(){
  const rlim_t ks=64*1024*1024;
  struct rlimit rl;
  int res=getrlimit(RLIMIT_STACK,&rl);
  if(!res&&rl.rlim_cur<ks){
    rl.rlim_cur=ks;
    res=setrlimit(RLIMIT_STACK,&rl);
  }
}
```

10.4 input

```
inline int read(){
  int x=0; bool f=0; char c=getchar();
  while(ch<'0'||'9'<ch)f|=ch=='-',ch=getchar
       ();
  while('0'<=ch&&ch<='9')x=x*10-'0'+ch,ch=
       getchar();
  return f?-x:x;
}
// #!/bin/bash
// g++ -std=c++11 -O2 -Wall -Wextra -Wno-
     unused-result -DDEBUG $1 && ./a.out
//  -fsanitize=address -fsanitize=undefined
     -fsanitize=return
```

11 other

11.1 WhatDay

```
int whatday(int y,int m,int d){
  if(m<=2)m+=12,--y;
  if(y<1752||y==1752&&m<9||y==1752&&m==9&&d
       <3)
    return (d+2*m+3*(m+1)/5+y+y/4+5)%7;
  return (d+2*m+3*(m+1)/5+y+y/4-y/100+y/400)
       %7;
}
```

11.2 上下最大正方形

```
void solve(int n,int a[],int b[]){// 1-base
  int ans=0;
  deque<int>da,db;
  for(int l=1,r=1;r<=n;++r){
    while(da.size()&&a[da.back()]>=a[r]){
      da.pop_back();
    }
    da.push_back(r);
    while(db.size()&&b[db.back()]>=b[r]){
      db.pop_back();
    }
    db.push_back(r);
    for(int d=a[da.front()]+b[db.front()];r-
         l+1>d;++l){
      if(da.front()==l)da.pop_front();
      if(db.front()==l)db.pop_front();
      if(da.size()&&db.size()){
        d=a[da.front()]+b[db.front()];
      }
    }
    ans=max(ans,r-l+1);
  }
  printf("%d\n",ans);
}
```

11.3 最大矩形

```
LL max_rectangle(vector<int> s){
  stack<pair<int,int > > st;
  st.push(make_pair(-1,0));
  s.push_back(0);
  LL ans=0;
  for(size_t i=0;i<s.size();++i){
    int h=s[i];
    pair<int,int > now=make_pair(h,i);
    while(h<st.top().first){
      now=st.top();
      st.pop();
      ans=max(ans,(LL)(i-now.second)*now.
           first);
    }
    if(h>st.top().first){
```

```
      st.push(make_pair(h,now.second));
    }
  }
  return ans;
}
```

12 other language

12.1 java

12.1.1 文件操作

```
import java.io.*;
import java.util.*;
import java.math.*;
import java.text.*;

public class Main{

  public static void main(String args[]){
       throws FileNotFoundException,
       IOException
    Scanner sc = new Scanner(new FileReader(
         "a.in"));
    PrintWriter pw = new PrintWriter(new
        FileWriter("a.out"));
    int n,m;
    n=sc.nextInt();//读入下一个INT
    m=sc.nextInt();

    for(ci=1; ci<=c; ++ci){
      pw.println("Case #"+ci+": easy for
          output");
    }

    pw.close();//关闭流并释放，这个很重要，
        否则是没有输出的
    sc.close();//关闭流并释放
  }
}
```

12.1.2 优先队列

```
PriorityQueue queue = new PriorityQueue( 1,
    new Comparator(){
  public int compare( Point a, Point b ){
  if( a.x < b.x || a.x == b.x && a.y < b.y )
    return -1;
  else if( a.x == b.x && a.y == b.y )
    return 0;
  else return 1;
  }
});
```

12.1.3 Map

```
Map map = new HashMap();
map.put("sa","dd");
String str = map.get("sa").toString;

for(Object obj : map.keySet()){
  Object value = map.get(obj );
}
```

12.1.4 sort

```
static class cmp implements Comparator{
  public int compare(Object o1,Object o2){
  BigInteger b1=(BigInteger)o1;
  BigInteger b2=(BigInteger)o2;
  return b1.compareTo(b2);
  }
}
public static void main(String[] args)
    throws IOException{
  Scanner cin = new Scanner(System.in);
  int n;
  n=cin.nextInt();
  BigInteger[] seg = new BigInteger[n];
  for (int i=0;i<n;i++)
  seg[i]=cin.nextBigInteger();
  Arrays.sort(seg,new cmp());
}
```

12.2 python heap

```
import heapq

heap = [7,1,2,2]
heapq.heapify(heap)
print(heap) # [1, 2, 2, 7]
heapq.heappush(heap, 5)
print(heap) # [1, 2, 2, 7, 5]
print(heapq.heappop(heap)) # 1
print(heap) # [2, 2, 5, 7]
```

12.3 python input

```
ans = sum(map(float, input().split()))
# input: 1.1 2.2 3.3 4.4 5.5
print(ans) # 16.5

(n, m) = map(int, input().split()) # 300 200
print(n * m) # 60000

Arr = list(map(int, input().split()))
# input: 1 2 3 4 5
print(Arr) # [1, 2, 3, 4, 5]
```

12.4 python output

```
hello = 'Hello'
world = 7122
print(f'{hello} {world}') # Hello 7122

import math
print(f'PI is approximately {math.pi:.3f}.')
# PI is approximately 3.142.

print('AAA {} BBB "{}!"'.format('Jin', 'Kela
    '))
# AAA Jin BBB "Kela!"

hello = 'hello, world\n'
hellos = repr(hello)
print(hellos) # 'hello, world\n'

x = 32.5
y = 40000
print(repr((x, y, ('spam', 'eggs'))))
# "(32.5, 40000, ('spam', 'eggs'))"

x = 7
print(eval('3 * x')) # 21
```

13 zformula

13.1 formula

13.1.1 Pick 公式

給定頂點坐標均是整點的簡單多邊形，面積 = 內部格點數 + 邊上格點數/2-1

13.1.2 圖論

1. 對於平面圖，$F = E - V + C + 1$，C 是連通分量數
2. 對於平面圖，$E \leq 3V - 6$
3. 對於連通圖 G，最大獨立點集的大小設為 I(G)，最大匹配大小設為 M(G)，最小點覆蓋設為 Cv(G)，最小邊覆蓋設為 Ce(G)。對於任意連通圖：
   (a) $I(G) + Cv(G) = |V|$
   (b) $M(G) + Ce(G) = |V|$
4. 對於連通二分圖：
   (a) $I(G) = Cv(G)$
   (b) $M(G) = Ce(G)$
5. 最大權閉合圖：
   (a) $C(u, v) = \infty, (u, v) \in E$
   (b) $C(S, v) = W_v, W_v > 0$
   (c) $C(v, T) = -W_v, W_v < 0$
   (d) ans=$\sum_{W_v>0} W_v - flow(S, T)$
6. 最大密度子圖：
   (a) 求 $max\left(\frac{W_e+W_v}{|V'|}\right), e \in E', v \in V'$
   (b) $U = \sum_{v \in V} 2W_v + \sum_{e \in E} W_e$
   (c) $C(u, v) = W_{(u,v)}, (u, v) \in E$，雙向邊
   (d) $C(S, v) = U, v \in V$
   (e) $D_u = \sum_{(u,v) \in E} W_{(u,v)}$
   (f) $C(v, T) = U + 2g - D_v - 2W_v, v \in V$
   (g) 二分搜 g：
   $l = 0, r = U, eps = 1/n^2$
   if$((U \times |V| - flow(S, T))/2 > 0)\, l = mid$
   else $r = mid$
   (h) ans=$min\_cut(S, T)$
   (i) $|E| = 0$ 要特殊判斷
7. 弦圖：
   (a) 點數大於 3 的環都要有一條弦
   (b) 完美消除序列從後往前依次給每個點染色，給每個點染上可以染的最小顏色
   (c) 最大團大小 = 色數
   (d) 最大獨立集: 完美消除序列從前往後能選就選
   (e) 最小團覆蓋: 最大獨立集的點和他延伸的邊構成
   (f) 區間圖是弦圖
   (g) 區間圖的完美消除序列: 將區間按造又端點由小到大排序
   (h) 區間圖染色: 用線段樹做

13.1.3 dinic 特殊圖複雜度

1. 單位流：$O\left(min\left(V^{3/2}, E^{1/2}\right) E\right)$
2. 二分圖：$O\left(V^{1/2} E\right)$

13.1.4 0-1 分數規劃

$x_i = \{0, 1\}$，x_i 可能會有其他限制，求 $max\left(\frac{\sum B_i x_i}{\sum C_i x_i}\right)$

1. $D(i, g) = B_i - g \times C_i$
2. $f(g) = \sum D(i, g) x_i$
3. $f(g) = 0$ 時 g 為最佳解，$f(g) < 0$ 沒有意義
4. 因為 $f(g)$ 單調可以二分搜 g
5. 或用 Dinkelbach 通常比較快

```
binary_search(){
  while(r-l>eps){
    g=(l+r)/2;
    for(i:所有元素)D[i]=B[i]-g*C[i];//D(i,g)
    找出一組合法x[i]使f(g)最大;
    if(f(g)>0) l=g;
    else r=g;
  }
  Ans = r;
}
Dinkelbach(){
  g=任意狀態(通常設為0);
  do{
    Ans=g;
    for(i:所有元素)D[i]=B[i]-g*C[i];//D(i,g)
    找出一組合法x[i]使f(g)最大;
    p=0,q=0;
    for(i:所有元素)
```

```
        if(x[i])p+=B[i],q+=C[i];
    g=p/q;//更新解，注意q=0的情況
  }while(abs(Ans-g)>EPS);
  return Ans;
}
```

13.1.5 學長公式

1. $\sum_{d|n} \phi(n) = n$
2. $g(n) = \sum_{d|n} f(d) \Rightarrow f(n) = \sum_{d|n} \mu(d) \times g(n/d)$
3. Harmonic series $H_n = \ln(n) + \gamma + 1/(2n) - 1/(12n^2) + 1/(120n^4)$
4. $\gamma = 0.57721566490153286060651209008240243104215$
5. 格雷碼 $= n \oplus (n >> 1)$
6. $SG(A + B) = SG(A) \oplus SG(B)$
7. 選轉矩陣 $M(\theta) = \begin{pmatrix} cos\theta & -sin\theta \\ sin\theta & cos\theta \end{pmatrix}$

13.1.6 基本數論

1. $\sum_{d|n} \mu(n) = [n == 1]$
2. $g(m) = \sum_{d|m} f(d) \Leftrightarrow f(m) = \sum_{d|m} \mu(d) \times g(m/d)$
3. $\sum_{i=1}^{n} \sum_{j=1}^{m}$ 互質數量 $= \sum \mu(d) \lfloor \frac{n}{d} \rfloor \lfloor \frac{m}{d} \rfloor$
4. $\sum_{i=1}^{n} \sum_{j=1}^{n} lcm(i,j) = n \sum_{d|n} d \times \phi(d)$

13.1.7 排組公式

1. k 卡特蘭 $\frac{C_n^{kn}}{n(k-1)+1} \cdot C_m^n = \frac{n!}{m!(n-m)!}$
2. $H(n,m) \cong x_1 + x_2 \ldots + x_n = k, num = C_k^{n+k-1}$
3. Stirling number of 2^{nd},n 人分 k 組方法數目
   (a) $S(0,0) = S(n,n) = 1$
   (b) $S(n,0) = 0$
   (c) $S(n,k) = kS(n-1,k) + S(n-1,k-1)$
4. Bell number,n 人分任意多組方法數目
   (a) $B_0 = 1$
   (b) $B_n = \sum_{i=0}^{n} S(n,i)$
   (c) $B_{n+1} = \sum_{k=0}^{n} C_k^n B_k$
   (d) $B_{p+n} \equiv B_n + B_{n+1} mod p$, p is prime
   (e) $B_{p^m+n} \equiv mB_n + B_{n+1} mod p$, p is prime
   (f) From B_0 : $1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, 115975$
5. Derangement, 錯排, 沒有人在自己位置上
   (a) $D_n = n!(1 - \frac{1}{1!} + \frac{1}{2!} - \frac{1}{3!} \ldots + (-1)^n \frac{1}{n!})$
   (b) $D_n = (n-1)(D_{n-1} + D_{n-2}), D_0 = 1, D_1 = 0$
   (c) From D_0 : $1, 0, 1, 2, 9, 44, 265, 1854, 14833, 133496$
6. Binomial Equality
   (a) $\sum_k \binom{r}{m+k}\binom{s}{n-k} = \binom{r+s}{m+n}$
   (b) $\sum_k \binom{l}{m+k}\binom{s}{n+k} = \binom{l+s}{l-m+n}$
   (c) $\sum_k \binom{l}{m+k}\binom{s+k}{n}(-1)^k = (-1)^{l+m}\binom{s-m}{n-l}$
   (d) $\sum_{k \le l} \binom{l-k}{m}\binom{s}{k-n}(-1)^k = (-1)^{l+m}\binom{s-m-1}{l-n-m}$
   (e) $\sum_{0 \le k \le l} \binom{l-k}{m}\binom{q+k}{n} = \binom{l+q+1}{m+n+1}$
   (f) $\binom{r}{k} = (-1)^k \binom{k-r-1}{k}$
   (g) $\binom{r}{m}\binom{m}{k} = \binom{r}{k}\binom{r-k}{m-k}$
   (h) $\sum_{k \le n} \binom{r+k}{k} = \binom{r+n+1}{n}$
   (i) $\sum_{0 \le k \le n} \binom{k}{m} = \binom{n+1}{m+1}$
   (j) $\sum_{k \le m} \binom{m+r}{k} x^k y^k = \sum_{k \le m} \binom{-r}{k}(-x)^k (x+y)^{m-k}$

13.1.8 冪次, 冪次和

1. $a^b \% P = a^{b \% \varphi(p) + \varphi(p)}, b \ge \varphi(p)$
2. $1^3 + 2^3 + 3^3 + \ldots + n^3 = \frac{n^4}{4} + \frac{n^3}{2} + \frac{n^2}{4}$
3. $1^4 + 2^4 + 3^4 + \ldots + n^4 = \frac{n^5}{5} + \frac{n^4}{2} + \frac{n^3}{3} - \frac{n}{30}$
4. $1^5 + 2^5 + 3^5 + \ldots + n^5 = \frac{n^6}{6} + \frac{n^5}{2} + \frac{5n^4}{12} - \frac{n^2}{12}$
5. $0^k + 1^k + 2^k + \ldots + n^k = P(k), P(k) = \frac{(n+1)^{k+1} - \sum_{i=0}^{k-1} C_i^{k+1} P(i)}{k+1}, P(0) = n + 1$
6. $\sum_{k=0}^{m-1} k^n = \frac{1}{n+1} \sum_{k=0}^{n} C_k^{n+1} B_k m^{n+1-k}$
7. $\sum_{j=0}^{m} C_j^{m+1} B_j = 0, B_0 = 1$
8. 除了 $B_1 = -1/2$，剩下的奇數項都是 0
9. $B_2 = 1/6, B_4 = -1/30, B_6 = 1/42, B_8 = -1/30, B_{10} = 5/66, B_{12} = -691/2730, B_{14} = 7/6, B_{16} = -3617/510, B_{18} = 43867/798, B_{20} = -174611/330,$

13.1.9 Burnside's lemma

1. $|X/G| = \frac{1}{|G|} \sum_{g \in G} |X^g|$
2. $X^g = t^{c(g)}$
3. G 表示有幾種轉法，X^g 表示在那種轉法下，有幾種是會保持對稱的，t 是顏色數，$c(g)$ 是循環節不動的面數。
4. 正立方體塗三顏色，轉 0 有 3^6 個元素不變，轉 90 有 6 種，每種有 3^3 不變，180 有 3×3^4，120(角) 有 8×3^2，180(邊) 有 6×3^3，全部 $\frac{1}{24}(3^6 + 6 \times 3^3 + 3 \times 3^4 + 8 \times 3^2 + 6 \times 3^3) = 57$

13.1.10 Count on a tree

1. Rooted tree: $s_{n+1} = \frac{1}{n} \sum_{i=1}^{n} (i \times a_i \times \sum_{j=1}^{\lfloor n/i \rfloor} a_{n+1-i \times j})$
2. Unrooted tree:
   (a) Odd:$a_n - \sum_{i=1}^{n/2} a_i a_{n-i}$
   (b) Even:$Odd + \frac{1}{2} a_{n/2}(a_{n/2} + 1)$
3. Spanning Tree
   (a) 完全圖 $n^n - 2$
   (b) 一般圖 (Kirchhoff's theorem)$M[i][i] = degree(V_i)$,$M[i][j] = -1$,if have $E(i,j)$,0 if no edge. delete any one row and col in A, $ans = det(A)$

14 Интернационал

14.1 ganadoQuote

```
¡Allí está!
¡Un forastero!
¡Agarrenlo!
¡Os voy a romper a pedazos!
¡Cógelo!
¡Te voy a hacer picadillo!
¡Te voy a matar!
¡Míralo, está herido!
¡Sos cerdo!
¿Dónde estás?
¡Detrás de tí, imbécil!
¡No dejes que se escape!
¡Basta, hijo de puta!
Lord Saddler...

¡Mátalo!
¡Allí está!
Morir es vivir.
Síííííí, ¡Quiero matar!
Muere, muere, muere....
Cerebros,cerebros,cerebros...
Cógedlo, cógedlo, cógedlo...
Lord Saddler...
Dieciséis.

¡Va por él!
¡Muérete!
¡Cógelo!
¡Te voy a matar!
¡Bloqueale el paso!
¡Te cogí!
¡No dejes que se escape!

¿Qué carajo estás haciendo aquí? ¡Lárgate, cabrón!
Hay un rumor de que hay un extranjero entre nosotros.
Nuestro jefe se encargará de la rata.
Su "Las Plagas" es mucho mejor que la nuestra.
Tienes razón, es un hombre.
Usa los músculos.
Se vuelve loco!
¡Hey, acá!
¡Por aquí!
¡El Gigante!
¡Del Lago!
¡Cógelo!
¡Cógenlo!
¡Allí!
¡Rápido!
¡Empieza a rezar!
¡Mátenlos!
¡Te voy a romper en pedazos!
¡La campana!
Ya es hora de rezar.
Tenemos que irnos.
¡Maldita sea, mierda!
¡Ya es hora de aplastar!
¡Mierda!
¡Puedes correr, pero no te puedes esconder!
¡Sos cerdo!
¡Está en la trampa!
¡Ah, que madre!
¡Vámonos!
¡Ándale!
¡Cabrón!
¡Coño!
¡Agárrenlo!
Cógerlo, Cógerlo...
¡Allí está, mátalo!
¡No dejas que se escape de la isla vivo!
¡Hasta luego!
¡Rápido, es un intruso!
```

14.2 Интернационал

```
/**************************
L'Internationale,
     Sera le genre humain.

            -. .
       _____    ',' ,
     ,'      ,'    ',  ',
   ,'      ,'       |  |
  \       \        |  |
    \ /^\   \    ,'  ,'
         \   \ ,' ,'
    / ~-.___\.-'  ,'
  /   .______.- ~ \
/   /'          \  \
\./              \/'
**************************/
Вставай, проклятьем заклеймённый,
Весь мир голодных и рабов!
Кипит наш разум возмущённый
И в смертный бой вести готов.
Весь мир насилья мы разрушим
До основанья, а затем
Мы наш, мы новый мир построим, —
Кто был ничем, тот станет всем.

Chorus
Это есть наш последний
И решительный бой;
С Интернационалом
Воспрянет род людской!

Никто не даст нам избавленья:
Ни бог, ни царь и не герой!
Добьёмся мы освобожденья
Своею собственной рукой.
Чтоб свергнуть гнёт рукой умелой,
Отвоевать своё добро, —
Вздувайте горн и куйте смело,
```

```
Пока железо горячо!

Chorus

Довольно кровь сосать, вампиры,
Тюрьмой, налогом, нищетой!
У вас — вся власть, все блага мира,
А наше право — звук пустой !
Мы жизнь построим по-иному –
И вот наш лозунг боевой:
Вся власть народу трудовому!
А дармоедов всех долой!

Chorus

Презренны вы в своём богатстве,
Угля и стали короли!
Вы ваши троны, тунеядцы,
На наших спинах возвели.
Заводы, фабрики, палаты –
Всё нашим создано трудом.
Пора! Мы требуем возврата
Того, что взято грабежом.

Chorus

Довольно королям в угоду
Дурманить нас в чаду войны!
Война тиранам! Мир Народу!
Бастуйте, армии сыны!
Когда ж тираны нас заставят
В бою геройски пасть за них –
Убийцы, в вас тогда направим
Мы жерла пушек боевых!

Chorus

Лишь мы, работники всемирной
Великой армии труда,
Владеть землёй имеем право,
Но паразиты — никогда!
И если гром великий грянет
Над сворой псов и палачей, –
Для нас всё так же солнце станет
Сиять огнём своих лучей.

Chorus
```

14.3 保佑

```
//                  _oo0oo_
//                 o8888888o
//                 88" . "88
//                 (| -_- |)
//                 0\  =  /0
//               ___/`---'\___
//             .' \\|     |// '.
//            / \\|||  :  |||// \
//           / _||||| -:- |||||- \
//          |   | \\\  -  /// |   |
//          | \_|  ''\---/''  |_/ |
//          \  .-\__  '-'  ___/-. /
//        ___'. .'  /--.--\  `. .'___
//     ."" '<  `.___\_<|>_/___.' >' "".
//    | | :  `- \`.;`\ _ /`;.`/ - ` : | |
//    \  \ `_.   \_ __\ /__ _/   .-` /  /
// ===`-.____`.___ \_____/___.-`___.-'===
//                 `=---='
//
// ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
//           佛祖保佑         永無BUG




















#    ┌─┐       ┌─┐
#  ┌─┘ └───────┘ └──┐
#  │                │
#  │      ───       │
#  │ ─┬┘       └┬─  │
#  │                │
#  │      ─┴─       │
#  │                │
#  └───┐        ┌───┘
#      │        │
#      │        │
#      │        │
#      │        └──────────────┐
#      │                       │
#      │                       ├─┐
#      │                       ┌─┘
#      │                       │
#      └─┐  ┐  ┌───────┬──┐  ┌──┘
#        │ ─┤ ─┤       │ ─┤ ─┤
#        └──┴──┘       └──┴──┘
#          神獸保佑  永無BUG!


// ##            ##################
// ##            ##
// ##            ##
// ##            ##
// ##            ##
// ##            ##
// ##            ##
// ##            ##
// ################################
//               ##              ##
//               ##              ##
//               ##              ##
//               ##              ##
//               ##              ##
//               ##              ##
//               ##              ##
// ################              ##
//
//        元首保佑  永無BUG

//                 __
// _____ __ __   < @ \    _  __ _____
//  \____\\_\\_)  /**|   (_//_//____/
//   \____\(_)\*\/****\/*/(_)/____/
//    \____\_/_\*\****/*/_\_/____/
//          u/u/u|****|u\u\u
//           u/u/|****|\u\u
//               |*||*|
//                |  |
//               /+--+\
//              ||  卐  ||
//               \\==//
//
//          神獸保佑  永無BUG
```

ACM ICPC Team Reference - CCU-oshiete536senpai

Contents

ACM ICPC Judge Test - CCU-oshiete536senpai

C++ Resource Test

```cpp
#include <bits/stdc++.h>
using namespace std;

namespace system_test {

const size_t KB = 1024;
const size_t MB = KB * 1024;
const size_t GB = MB * 1024;

size_t block_size, bound;
void stack_size_dfs(size_t depth = 1) {
  if (depth >= bound)
    return;
  int8_t ptr[block_size]; // 若無法編譯將 block_size 改成常數
  memset(ptr, 'a', block_size);
  cout << depth << endl;
  stack_size_dfs(depth + 1);
}

void stack_size_and_runtime_error(size_t block_size, size_t bound = 1024) {
  system_test::block_size = block_size;
  system_test::bound = bound;
  stack_size_dfs();
}

double speed(int iter_num) {
  const int block_size = 1024;
  volatile int A[block_size];
  auto begin = chrono::high_resolution_clock::now();
  while (iter_num--)
    for (int j = 0; j < block_size; ++j)
      A[j] += j;
  auto end = chrono::high_resolution_clock::now();
  chrono::duration<double> diff = end - begin;
  return diff.count();
}

void runtime_error_1() {
  // Segmentation fault
  int *ptr = nullptr;
  *(ptr + 7122) = 7122;
}

void runtime_error_2() {
  // Segmentation fault
  int *ptr = (int *)memset;
  *ptr = 7122;
}

void runtime_error_3() {
  // munmap_chunk(): invalid pointer
  int *ptr = (int *)memset;
  delete ptr;
}

void runtime_error_4() {
  // free(): invalid pointer
  int *ptr = new int[7122];
  ptr += 1;
  delete[] ptr;
}

void runtime_error_5() {
  // maybe illegal instruction
  int a = 7122, b = 0;
  cout << (a / b) << endl;
}

void runtime_error_6() {
  // floating point exception
  volatile int a = 7122, b = 0;
  cout << (a / b) << endl;
}

void runtime_error_7() {
  // call to abort.
  assert(false);
}

} // namespace system_test

#include <sys/resource.h>
void print_stack_limit() { // only work in Linux
  struct rlimit l;
  getrlimit(RLIMIT_STACK, &l);
  cout << "stack_size = " << l.rlim_cur << " byte" << endl;
}
```